Mechanizing Adjoint Logic

MAX GROSS, ASHA BASU, and ARMAN AHMED, McGill University, Canada

ACM Reference Format:
Max Gross, Asha Basu, and Arman Ahmed. 2026. Mechanizing Adjoint Logic. 1, 1 (March 2026), 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Substructural logics, those logics that remove some structural assumptions made in intuitionistic systems, have long been of interest in computation. They are of particular note in context of modelling type systems, where their substructurality allows the formal guarantee of programmatic properties. With ever-increasing demands for verification across software systems, it is becoming increasingly important that we have logical systems that can adequately represent and reason about complex systems across programming language paradigms. Further, the scale and depth of these systems is quickly outgrowing the limitations that human-verified proofs have in relation to length and formal rigor.

Linear logic, as first proposed by Jean-Yves Girard in 1987, is a substructural logic that does not assume weakening or contraction, that is, it is sensitive to the number of times a proposition is used [5]. In order to recover expressibility, Girard introduced the ! operator that allow for a variable to be treated as 'resource agnostic' (as it would in intuitionistic logic) [5]. While this system does allow us the expressibility of intuitionistic logic, it presents a profoundly asymmetric view of logic, centering linearity and viewing intuitionistic logic as a mere extension.

Adjoint logic presents a more symmetric view of this picture, while extending the substructural constraints that we are able to represent. Adjoint logic allows the creation of modes, multiple substructural systems (substructural systems with varying allowed structural rules), with the $\uparrow_k^n$ and $\downarrow_n^k$ allowing for these systems to interact. This profoundly open structure allows for the encoding of numerous logical systems, including a fragment of intuitionistic S4, lax logic, and, of course, linear logic [7].

The generality of adjoint logic allows for the modelling of much more complex software systems, specifically those that operate between languages with different type systems. This property comes, unsurprisingly, from the $\uparrow_k^n$ and $\downarrow_n^k$, which allow us to have the type-theoretic guarantees that exist within different logical systems to effectively have interplay.

It is precisely because of this unifying nature that we face difficulty in trying to the normalization of this system. Existing normalization proofs for substructural systems often do not use the logical relations argument (which is particularly amenable for mechanization), instead relying on less general proof techniques that involve explicitly counting variables and graphical arguments. Further, existing normalization proofs of adjoint logics are

Authors' Contact Information: Max Gross, max.gross@mail.mcgill.ca; Asha Basu, asha.basu@mail.mcgill.ca; Arman Ahmed, arman.ahmed@mail.mcgill.ca, McGill University, Montréal, Canada.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2026/3-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

regarding specific implementations of adjoint logic, using specific modes that one can define within the adjoint framework [9].

As such, we present the following contributions:

- We present a novel weak normalization proof for a fully-generalized presentation of adjoint logic, as presented in [7].
- We show weak normalization with a full-β dynamic semantics.
- We employ a relatively new approximate-typing proof that allows us to painlessly employ a logical relations proof of normalization, which is much simpler than other techniques.
- We present a mechanization of these in the Beluga proof assistant, giving us a formal guarantee of the proof as well as a computational framework for future metatheoretic development.

2 Motivations

2.1 The difficulty of Logical Relations Proofs in Adjoint Logic

A logical relations proof of normalization, in abstract, allows us to prove normalization through its fundamental lemma: that all well-typed terms in a language are normalizing under any substitution of that satisfies the predicate [9]. In order to prove this, we define families of typed terms such that any term within this family satisfies this condition [9]. While in the intuitionistic setting, where contexts maintain relative stability, this does not present significant challenges for proof, we find that this does not hold with substructural, let alone adjoint logics [9] [1].

An illustrative example of this is Girard's approach [5]. While he uses an argument similar in structure to a standard logical relations proof, it is applied to the proof net presentation of linear logic. We find that this system does not have easy transferrability to natural deduction presentations because of the syntactic guarantees that are present in the definition of proof nets (which are not present in a natural deduction system) [5]. Among the most significant results of this is that, in his proof of strong normalization, he is able to avoid proof (as it comes from the definition of proof nets) of the context-substitution stipulation of the fundamental lemma, which is where the necessity of approximate types arrives.

There are also proofs of cut-elimination in the sequent-calculus presentations of adjoint logic, most notably in [11]. These proofs illustrate a similar proof-theoretic property to normalization proofs (that proofs in the system are terminating), though sequent-calculi have more muddy relations to type-systems as compared to natural deduction presentations.

[7] uses an intriguing method for proving meta-theoretic properties, including soundness and completeness, by showing a bi-directional correspondence between terms in the sequent-calculus and those in natural deduction. While this method seems promising for proving normalization of their system, it is much harder to mechanize because of the structure of the sequent-calculus.

2.2 Advantages of the Logical Relations Technique

We find utility in the logical relations method, then, precisely because of its increased generality, extensionality, and ease mechanizations that the examples outlined above lack. Logical relations proofs have a remarkably simple logical flow: We define sets of terms that display favorable properties, show that all well-typed terms fit into one of these families, prove that these families are normalizing, and therefore conclude that all well-typed terms are normalizing. This structure maintains a much higher level of generality than other methods precisely because it deals only with the terms (and the derivations of those terms) themselves, allowing these proofs to be easily transferred to analogous systems. Further, because their stucture is based on these reducibility candidates (referred to as type-families above), proving normalization for extensions of a given system is remarkably simple; all that is required is to create more reducibility candidates for each family, then prove the properties described

above for them. Further, because these proofs deal only with terms, they rely solely on structural induction on the size of these terms, allowing for clean mechanizations as compared to other methods.

2.3 Full-β Reduction

[9] employs an extension of call-by-value semantics in its language. This allows them to avoid proving the Church-Rossier theorem as their semantics are deterministic. We find particular interest in a full-β dynamic semantics because it allows our proof of weak normalization to rely purely on the typing constraints and static semantics of the system, making our proof more easily generalizable to future applications. Similarly, this better illustrates the power of the approximate-typing technique pioneered in [9].

2.4 Generalized Adjoint Logic

We, perhaps, find the most compelling motivation of our proof in its generality. As discussed previously, adjoint logic allows for the unification of any arbitrary amount of logical systems under a unified system that allows for interoperability. As such, the previously identified proofs that deal in fragments of the system must be reproduced for every new fragments. Since our proof allows for adjoint logic to be deployed in any of its forms (with an arbitrary amount and character of the logics it unites), we see large applicability of our proof to any future projects in adjoint logical type systems.

2.5 Advantages of Mechanization

The difficulties that stem from this, of course, is in proof length. The demands of a fully-generalized proof of a system as large as adjoint logic requires significant case analysis to prove structural properties. Combined with the logical precision required to prove such properties on programming languages, we find that mechanization is a natural step both in its correctness guarantees and in the transferrability or structural induction proofs from paper to proof-assistant. The Beluga proof assistant is particularly apt for our use case due to its explicit handling of contexts, allowing for easier encoding of many of the propositions we prove, as well as its particular aptitude in encoding deductive systems.

3 Main Contributions

3.1 Static semantics

First, we mechanize a static semantics for adjoint logic. To our knowledge, there is no preexisting mechanization of adjoint logic in the literature, so this is the first such. To this end, we mechanize a variant of the Adjoint Natural Deduction system outlined by Jang et al. [7]. In particular, we drop the bidirectional typing they use, and instead just mechanize typing judgments of the form $e_m : A_m$. We also allow weakening at any point than allowing it only in the rules for variable type-checking and mode-switching, as this was more amenable to mechanization with the method we used.

In our formalization and subsequent mechanization, our terms are not intrinsically typed, but both our terms and our types are intrinsically *moded*; types, terms, and connectives are all indexed by mode. Thus in any inference rule, we implicitly quantify over modes. The adjoint logic we mechanize also allows for an arbitrary partial order of modes, and assumes that each mode allows for every connective. In our mechanization, we allow for the assertion of arbitrary relations between modes and arbitrary structural properties of these modes, and our proofs all follow regardless.

In order to ensure that our mechanization follows the substructurality constraints, we use Crary's Higher-order Representation of Structural Logics [3]. The idea is that each time a variable is introduced, such as in the typing judgment for a lambda, we need to verify that the LF function that gives us a lambda term uses the variable we are binding in a way that obeys the substructurality constraints. To this end, we define a predicate observe that,

given a term, asserts that it obeys the substructurality constraints. The predicate is then built up inductively from assertions of the predicate on different subterms of the term we wish to verify, where we use the fact that if a variable is not explicitly stated to occur in some subterm then it cannot, since metavariables that are not explicitly stated to have a dependency on some other variable are assumed to be bound from the outside in Beluga.

Derivations in adjoint logic also require at points that we have $m \geq n$ for two modes m and n, such as in pattern-matching. We thus pass witnesses/assertions that one mode is above another to typing derivations as premises. Additionally, in adjoint logic, for a derivation of the form $\Gamma \vdash e_m : A_m$, we presuppose that all variables $x : B_n \in \Gamma$ have $n \geq m$; we can only use variables of modes greater than equal to the current mode in a derivation, or else it would be possible to bypass the substructurality. Thus in our observe predicate, we also verify that the LF function uses its input in a way that respects this mode constraint. To this end, we pass to observe another parameter that witnesses $m \geq n$ whenever the LF function we wish to verify takes terms of mode m to terms of mode n. We can then pass this parameter in all our constructors that do not change mode by taking the witness parameter from the subterm we've verified; for example, if observe _ $\lambda x.\langle M\ x, N\rangle$ is constructed from observe _ $\lambda x.M\ x$ (where $M\ x$ denotes that x occurs in M), then we can take the first parameter to the observe predicate of $M\ x$ as our first parameter for the observe predicate of the pair.

3.2 Dynamic semantics

We also mechanize a full β dynamics (without η or commuting conversions) for this adjoint logic system; to our knowledge, this dynamics differs from all existing dynamics in the literature, such as the dynamics in Jang et al. [7]. We mechanize both a single-step ($\longrightarrow$) and parallel reduction ($\twoheadrightarrow$) relation, prove confluence for the latter, and show that their reflexive transitive closures are equivalent proving confluence for the former as well.

Since our terms are intrinsically moded, mode preservation is inherent to the dynamics; we do not show subject reduction, since it is not used anywhere in our proof. Furthermore, the dynamics does not interface with the substructurality at all; as a rewriting system, the dynamics at no point requires that any substructurality constraint is satisfied, which motivates us to use the approximate typing technique pioneered in [9].

3.3 Approximate types

We use approximate types; for us, since terms and types are intrinsically moded, our approximate typing is just exactly our regular typing but without the observe predicate; thus it is straightforward to mechanize the proof that well-typed terms are also approximately well-typed. We can then carry out a logical relations-style normalization proof, with the details much the same as (but more general than) [9].

In particular, the approximate types are used in the proof of the fundamental lemma. In this proof, we need the contexts in the premises to be roughly the same (modulo the addition of one or two variables), which means we can't have any sort of context splitting; approximate types allow us to work with a structural system and thereby avoid this problem, while still remaining fine-grained enough to prove the fundamental lemma, since the information we're using (substructurality) wasn't relevant to the dynamics.

3.4 Proof of normalization

The bulk of our contribution is a proof of normalization. We mostly follow Kavanagh et al. in the structure of our proof, but adapt our proof to deal with the new connectives and arbitrary families of modes[9]. They also use deterministic reductions, which make some of the lemmas much easier to prove; having to work with full β increases the work needed to be done. Here we outline the steps of our proof, presented especially in contrast to their work since our proof mostly follows theirs:

3.4.1 Definitions. We define normal and neutral terms in mostly the same way as Jang et al. [7]. The exception is that we make pattern-matches neutral (synthesizable, in their terms). This is in contrast to Kavanagh et al.,

who define canonical forms as well for a different structure of normal forms[9]. We also define reducibility candidates in mostly the same way, extending the definitions to additive connectives, except we (due to our different definition of normal/neutral terms) do not need a separate rule for pattern-matches, instead handling those as neutral terms. This involves an extensional definition for lambdas and the up-arrow and an intensional definition for the other constructors, and rules for backwards closure (under single reductions) and neutral terms. We take less care than they do to lay out the definitions of different sorts of contexts in the written-up proof, but follow the same approach in our mechanization, and we define reducible substitutions in much the same way, requiring that the terms being substituted are normal.

3.4.2 Lemmas about dynamics. We first prove a lemma that normal and neutral terms do not step to any other terms; this is intuitively a desirable trait for these definitions, but it needs to be proved for a lemma later on. The lemma follows by a basic inductive argument on the structure of the terms, where we examine the possible rules by which we can conclude that some normal or neutral term steps, conclude that it must be a congruence, conclude that some normal of neutral subterm must step to some other term, and conclude contradiction by the inductive hypothesis. Thus normal and neutral terms do indeed satisfy the expected properties.

We next need the diamond property for parallel reductions. We verified it by examining the cases briefly by hand, but didn't have the time to write up the full proof; the proof is, however, relatively straightforward and goes the same way as any similar proof does in any natural deduction system. We then use the standard argument to extend this to confluence for the reflexive transitive closure of parallel reductions. We additionally prove that substitutions do not change either single-step or parallel reductions, to prove this, we first prove reflexivity for parallel reductions, and then induct on the derivation of either reduction in the naive way to conclude this, though .

We then prove that the reflexive transitive closures of single-step and parallel reductions are identical; we do so by showing that any single-step reduction is a parallel reduction, thus implying the same statement for their reflexive transitive closures, and showing that any parallel reduction is in the reflexive transitive closure of single-step reductions, allowing us to chain together these chains to make a chain of parallel reductions into a chain of single-step reductions. Thus we can also conclude confluence for the reflexive transitive closure of single-step reductions, which is what we actually use, as our definitions (of eg our reducibility candidates) are made with reference to single-step reductions.

These proofs are only partly mechanized; we didn't get around (in the project's timespan) to finishing the mechanization of these proofs.

3.4.3 Lemmas about reducibility candidates. First, we prove (in Lemmas 4 and 7) that our reducibility candidates are closed under backwards closure and forwards reduction of chains of reductions (ie they are a coarsening of β-equivalence). The former proof proceeds by repeatedly applying backwards closure for single steps to the chain of reductions. For the latter proof, we know that neutral terms and units can't step to anything else, for backwards closure we can just invoke confluence and the inductive hypothesis, and for most constructors we can perform a straightforward induction on the chain of reductions, so the only interesting case is for lambdas. For lambdas, halting follows by confluence, but the other condition is more subtle and only follows due to the earlier lemma about substitutions preserving reductions, where we must repeatedly via congruences lift the given chain of reductions to the application and then conclude via the inductive hypothesis. This pattern will continue, where the most difficult case is often the case for lambdas, made yet more difficult by the fact that we allow reductions under lambdas (whereas Kavanagh et al. do not)[9].

Next, we prove that if $\Gamma \vdash \text{force } M$ halts, then so does $\Gamma \vdash M$. This is a straightforward induction on the chain of reductions by which it halts; if this is already normal then we know M is as well due to the structure of normal terms. Otherwise, we know that force M can only step via congruence relation or β-reduction. In the former

case, we can use the premise to step M forward and then apply the IH; in the latter case, we need to show that susp M' halts if M' does and can achieve this by repeated application of congruence rules.

Next, we prove that all terms in reducibility candidates halt. This follows by definition for some rules, and for the other by invoking the inductive hypothesis and repeatedly applying congruence rules, with no interesting ideas in the proof.

Next, we prove that neutral substitutions preserve reducibility of terms and substitutions; this follows by a very straightforward inductive argument in every case (composing substitutions in the case of lambdas) except for backwards closure, where we need to invoke the earlier lemma that substitutions preserve reductions.

Next, we prove the converse of the definitions for the reducibility candidates for lambdas and the up-arrow: that force and application preserve reducibility. The induction is straightforward, where we invoke the inductive hypothesis if the reducibility arose from backwards closure, examine the structure of neutral terms if the term is neutral, and otherwise conclude that the term is reducible by the very definition we seek to prove.

Finally, we also prove a similar lemma for projections, as we defined our reducibility candidates for additive conjunction via the constructors; this also follows straightforwardly in a similar way.

We managed to mechanize most of these lemmas, but not all of them again due to time constraints; however, all of these lemmas should be very straightforward to mechanize.

3.4.4 Approximate types and fundamental lemma. We prove that any well-typed term is approximately well-typed; this is extremely straightforward, and in the Beluga mechanization merely requires ignoring the observe predicate.

Finally, we prove the fundamental lemma, which implies (weak) normalization as a corollary. Most cases are straightforward or follow by the lemmas above, but we outline the cases for pattern-matching and lambdas, since those are the most interesting cases where we have to add variables to substitutions.

For lambdas, we require (as distinct from some logical relations proofs) that reducible terms halt, following Kavanagh et al. This means we need to prove that our lambda halts, which follows because the premise is reducible and thus halts by the inductive hypothesis. We also need to prove that it yields reducible terms when applied to reducible terms; to prove this, we first need to use that the input is reducible and thus halts in order to step it to a normal (and reducible) term which we can add to our reducible substitution before applying this substitution to the premise, yielding another reducible term which our application of the given lambda steps to via a β-reduction, proving the desired conclusion.

For pattern matches, we essentially apply the inductive hypothesis to the term being matched to conclude that (under the appropriate substitution) it is reducible and thus halts, and thus we can step it and the conclusion forward to some term such that the term being matched with is normal and reducible, so we can without loss of generality work with a normal term then. This rules out backwards closure as a candidate for why it is reducible, so either it follows by neutrality, in which case (invoking the various lemmas about substitutions) we can step the other terms to normal terms (since they are reducible and thus halt by the inductive hypothesis) and thus get that the desired term is neutral and thus reducible; or it follows by the definition of reducibility candidates for its constructor, so we can β-reduce the pattern to a term that (by the inductive hypothesis, using that the term being matched is normal and can be added to a reducible substitution) is reducible, finishing the proof.

We didn't end up mechanizing these lemmas in time; the fundamental lemma is likely the most difficult lemma to mechanize, but we expect that the proof written up is amenable to mechanization and should follow without too much difficulty.

4 Relevant Works

4.1 Adjoint Logical and Related Systems

Benton [1] pioneered the mode-based approach to unifying different logical systems through his introduction of linear/nonlinear logic. His paper gives comprehensive overview of his system, which represented both intuitionistic and linear logic, as well as possible applications of it to computation [1]. His framework was then generalized into the first presentation of adjoint logic, which was in the sequent calculus by Pfenning et al. [12]. Jang et al. adapted this system to create the adjoint natural deduction presentation that our project is built on [7].

Sano et al. [14] introduces a mechanization of session types through linear logic in Beluga. In their presentation, they give a proof of type-presentation for their system, paving the way for metatheoretic proofs of significant properties such as deadlock freedom [14].

Adjoint logical systems have seen wide-spread development in a variety of contexts. Hanukaey et al. [6] presents a generalization of a graded/linear dependent type system to support more general modes. Their project is of particular note for observing the ways in which adjoint logic's generality can greatly increase the power of Benton's linear/non-linear logic as they start their project working within said system, generating a two-layer system, before exploring its extensions within the adjoint program [6]. Licata and Shulman offer an extension of adjoint logic, cohesive homotopy type theory [10]. In particular, they replace the pre-order of modes given in adjoint logic with a 2-category, allowing for multiple morphisms to exist between two modes of the system, allowing their system to have wider applicability in category theoretic contexts [10]. Boyland and Pfenning [2] present a compilation between adjoint natural deduction and the semi-axiomatic sequent calculus, which is a promising step towards a compiler for adjoint-logical type systems. Finally, Kakutani et al. [8] presents a formulation of dual-context modal logic using an adjoint formulation with applications to staged computation.

4.2 Quantum Programming Languages and Other Applications

[9] developed Proto-Quipper-A which utilizes a two-mode adjoint logical system to generate a quantum functional and circuit description language. Besides being the inspiration for this paper, their work is also of note because of their mechanization of several of the meta-theoretic properties, including subject reduction, of their system in Beluga.

Pruiksma and Pfenning [13] describe a formulation of session types in adjoint logic which allow for the representation of several patterns in message-passing communication. While their formulation is based on the sequent-calculus presentation of adjoint logic, they prove several meta-theoretic properties of note to signal communication, including session fidelity and deadlock freedom [13]. Their system was later implemented into a language by Francalanza et al. [4].

5 Conclusion

We present a fully generalized normalization proof of an adjoint natural deduction system. In particular, our system employs a full-β dynamic semantics, necessitating a proof of confluence. We extend the approximate-typing technique used in [9] to a more broad system. Finally, we present a mechanization in Beluga to ensure the correctness and improve possible applications of our proof.

5.1 Future Work

We find the most obvious extension of our work in proving stronger facts about the normalization of our system. In particular, strong normalization under commuting conversions would be of great use in further guaranteeing the safety of adjoint logical systems. In this vein, metatheoretic proofs on extensions of the adjoint logical system, including those with additional modalies or recursive types would be a natural step, given the modular nature of logical relations proofs as used here.

The approximate-typing technique used is also of great interest in proving meta-theoretic facts about other substructural systems. As outlined above, this technique allows for much more tractable proofs of substructural systems with natural-deductive presentations by eliminating much of the complexity that comes from the context-splitting that is often inherent to their design. We hope that this technique can also allow for the easier mechanization of already existing metatheoretic proofs of substructural logics.

Though not directly related to the work done here, we see a longer term avenue in further applications of adjoint logical systems in software verification.

References

[1] P.N. Benton. [n. d.]. A mixed linear and non-linear logic: Proofs, terms and models. In *Lecture Notes in Computer Science*. Springer. https://link.springer.com/chapter/10.1007/bfb0022251 Springer Link.

[2] JOANNA BOYLAND and FRANK PFENNING. [n. d.]. Compiling adjoint natural deduction to the semi-axiomatic sequent calculus. ([n. d.]).

[3] Karl Crary. 2010. Higher-order representation of substructural logics. In *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming* (Baltimore, Maryland, USA) *(ICFP '10)*. Association for Computing Machinery, New York, NY, USA, 131–142. doi:10.1145/1863543.1863565

[4] Adrian Francalanza, Gerard Tabone, and Frank Pfenning. 2024. Implementing a Message-Passing Interpretation of the Semi-Axiomatic Sequent Calculus (Sax). In *International Conference on Coordination Models and Languages*. Springer, 295–313.

[5] Jean-Yves Girard. 1987. Linear logic. *Theoretical Computer Science* 50, 1 (1987), 1–101. doi:10.1016/0304-3975(87)90045-4

[6] Peter Hanukaev and Harley Eades III. 2023. Combining Dependency, Grades, and Adjoint Logic. In *Proceedings of the 8th ACM SIGPLAN International Workshop on Type-Driven Development* (Seattle, WA, USA) *(TyDe 2023)*. Association for Computing Machinery, New York, NY, USA, 58–70. doi:10.1145/3609027.3609408

[7] Junyoung Jang, Sophia Roshal, Frank Pfenning, and Brigitte Pientka. 2024. Adjoint Natural Deduction (Extended Version). arXiv:2402.01428 [cs.LO] https://arxiv.org/abs/2402.01428 Submitted on 2 Feb 2024.

[8] Yoshihiko Kakutani, Yuito Murase, and Yuichi Nishiwaki. 2019. Dual-context Modal Logic as Left Adjoint of Fitch-style Modal Logic. *Journal of Information Processing* 27 (2019), 77–86. doi:10.2197/ipsjjip.27.77

[9] Ryan Kavanagh, Chuta Sano, and Brigitte Pientka. 2025. Deconstructed Proto-Quipper: A Rational Reconstruction. arXiv:2510.20018 [cs.PL] https://arxiv.org/abs/2510.20018 Submitted to the 35th European Symposium on Programming (ESOP 2026).

[10] Daniel R Licata and Michael Shulman. 2015. Adjoint logic with a 2-category of modes. In *International Symposium on Logical Foundations of Computer Science*. Springer, 219–235.

[11] Frank Pfenning. 2018. Adjoint Logic. (2018). https://www.scs.cmu.edu/afs/cs.cmu.edu/Web/People/fp/papers/adjoint18b.pdf Manuscript, Carnegie Mellon University.

[12] Klaas Pruiksma. 2024. *Adjoint Logic with Applications*. Technical Report CMU-CS-24-103. Carnegie Mellon University, School of Computer Science. http://reports-archive.adm.cs.cmu.edu/anon/2024/CMU-CS-24-103.pdf Technical Report.

[13] Klaas Pruiksma and Frank Pfenning. 2021. A message-passing interpretation of adjoint logic. *Journal of Logical and Algebraic Methods in Programming* 120 (2021), 100637. doi:10.1016/j.jlamp.2020.100637

[14] Chuta Sano, Ryan Kavanagh, and Brigitte Pientka. 2023. Mechanizing Session-Types using a Structural View: Enforcing Linearity without Linearity. *Proc. ACM Program. Lang.* 7, OOPSLA2, Article 235 (Oct. 2023), 26 pages. doi:10.1145/3622810